BUSINESS REQUIREMENTS DOCUMENT

Multi-Agent Knowledge Assistant

Enterprise-grade Multi-Agent Knowledge Intelligence Platform

Version 2.0 (supersedes v1.0 of 29 October 2026) | **Date:** 30 October 2026

Technology: Python 3.12, FastAPI, LangGraph, LangChain-compatible tooling, OpenAI / Gemini (optional), PostgreSQL, Qdrant (production) / ChromaDB (development), Redis, React + TypeScript, Docker

Document purpose	Define the business goals, users, scope, functional and non-functional requirements, architecture, security, evaluation and acceptance criteria for the upgraded platform, and record - honestly - which requirements are implemented, how they were verified, and what remains.
How to read the status labels	**Implemented, tested**: implemented and covered by automated tests that were executed. **Implemented, partly tested**: implemented, but a dependency (live LLM API, GPU model, container runtime, external server) was not available in the verification environment, so only the stated part was tested. **Pending**: not implemented.

Revision history

Version	Date	Change
1.0	29 Oct 2026	Initial BRD: PDF/CSV/XLSX/DOCX/TXT/MD/JSON/URL ingestion, hybrid search, exact counts, table analysis, LangGraph routing.
2.0	30 Oct 2026	Enterprise upgrade: multi-tenant auth and RBAC, PostgreSQL/Qdrant/Redis architecture, background workers, advanced RAG, 8-agent supervisor graph with verification, document intelligence, knowledge graph, evaluation framework, React dashboard, security hardening, Docker/CI.

1. Executive Summary

The Multi-Agent Knowledge Assistant lets people upload large, mixed-format document collections (or point at public web pages), ask questions in natural language and receive concise answers backed by verifiable evidence - file, page, sheet and row, section, or URL. Version 1 delivered the core: multi-format ingestion, hybrid retrieval, exact full-corpus search with occurrence counts, controlled spreadsheet analysis and a LangGraph router. Version 2 turns it into a multi-tenant platform: authenticated users with roles, isolated tenants, durable background ingestion, a production data tier, an advanced retrieval pipeline, a supervisor with eight cooperating agents and a verification step, document intelligence, a measurable evaluation harness and a dashboard.

Design principles that shape every requirement

- **Evidence or abstention.** Every answer cites its sources; when evidence is insufficient the system says so instead of guessing.
- **Code computes, models explain.** Occurrence counts and spreadsheet figures are produced by deterministic code over the complete data; an LLM, when configured, only phrases grounded answers and may never execute code or SQL.
- **Documents are data, never instructions.** Retrieved text is fenced, scanned and filtered; tools are a closed whitelist.
- **Isolation by construction.** Tenant and workspace are part of every storage address; access control is enforced below the API layer.
- **Measure, don't assume.** Retrieval and answer quality, latency, tokens and cost are measured by a built-in evaluation harness.
- **Honest status.** This document distinguishes implemented-and-tested from untested and pending.

Verification summary (executed on 30 Oct 2026)

Suite	Result	Scope
Backend (pytest)	166 passed, 0 failed	ingestion, OCR, security, auth/RBAC/tenancy, RAG pipeline, agents, data ops, intelligence, evaluation, PostgreSQL, Redis, Qdrant (embedded), ChromaDB
Frontend (Vitest)	19 passed	API client (incl. SSE), chat, documents, login, role gating, report view, XSS escaping
End-to-end (Playwright)	12 / 12 steps passed	real server + built React app + headless Chromium: sign-up, upload, ask (semantic/exact/table), filters, history, evaluation, admin, viewer role
Production-shaped smoke	12 / 12 checks passed	separate API and worker processes, PostgreSQL (data + LangGraph checkpoints), Redis queue, CLI tenant bootstrap, production-mode config validation
Static checks	ruff clean; tsc strict clean	backend lint; frontend type-check and production build

Not verifiable in the build environment (see Appendix A): live OpenAI/Gemini calls, a Qdrant *server* (embedded mode was tested), the Docker build, GitHub Actions execution, RAGAS, a cross-encoder model.

2. Business Problem and Opportunity

- Large files hold hundreds or thousands of pages; manual review is slow and error-prone.
- Teams need one search across different formats (PDF, spreadsheets, Word, web) without opening each file.
- Purely semantic search misses exact names, IDs, dates and cannot count occurrences; spreadsheet questions need calculation, not similarity.
- Answers are only useful if users can verify where they came from; unverifiable answers are a compliance and trust risk.
- Enterprises require identity, role-based access, tenant isolation, audit trails, rate limiting and data deletion before they can adopt AI search on confidential documents.
- Operators need to know quality, latency and cost - and to detect regressions when models or settings change.

3. Business Objectives

- **BO-1** Answer plain-language questions over large mixed-format collections with citations (file, page, sheet/row, section, URL).
- **BO-2** Provide reliable exact search with complete occurrence counts and page/row locations over the entire extracted corpus.
- **BO-3** Provide correct, controlled calculations on CSV/Excel (filter, sort, group, aggregate, duplicates, comparison) without executing generated code.
- **BO-4** Deliver document intelligence: summaries, cross-document comparison, entity and table extraction, question generation, follow-ups, multilingual retrieval, knowledge graph.
- **BO-5** Operate as a secure multi-tenant service: JWT authentication, RBAC, document-level access control, audit logging, rate limiting, deletion.
- **BO-6** Scale ingestion and search through background workers, PostgreSQL, Redis and Qdrant.
- **BO-7** Make quality measurable (Recall@K, Precision@K, MRR, faithfulness, answer relevancy, latency, tokens, cost) and expose it in a dashboard.
- **BO-8** Make uncertainty and missing evidence visible: confidence levels, limitations, and explicit 'could not verify' responses.

4. Stakeholders and Users

Stakeholder / role	Needs	Platform role
End user / analyst	Ask questions, check evidence, export findings	viewer (ask, read) or editor (also upload, re-index, delete own)
Knowledge manager	Curate sources, share or restrict documents, evaluate quality	editor; document visibility 'organisation' or 'private'
Tenant administrator	Manage users and roles, review audit log, settings, erase data	admin
Platform operator / developer	Deploy, configure providers and stores, monitor, upgrade	environment configuration, CLI, worker, CI/CD
Security / compliance reviewer	Review isolation, audit trail, retention and deletion, injection and SSRF defences	read-only review of this BRD, audit log, security tests
Business owner	Priorities, acceptable cost, success measures	evaluation dashboard (accuracy, latency, cost/query)

5. Scope

5.1 In scope (v2)

- Multi-format ingestion with OCR; background ingestion jobs with progress; re-index and delete.
- Advanced RAG, eight-agent LangGraph workflow, verification and bounded retries, conversational memory, streaming.

- Structured data analysis; document intelligence; knowledge graph and Graph RAG; image files via OCR and optional vision captions; voice queries.
- Authentication, RBAC, multi-tenancy, audit, rate limiting, data deletion; PostgreSQL, Redis, Qdrant/ChromaDB; Docker and CI/CD; React dashboard; evaluation harness and reports.

5.2 Out of scope / explicitly not claimed

- Guaranteed accuracy of OCR on poor scans or handwriting; understanding of images embedded inside PDFs (only standalone image files are indexed).
- Autonomous execution of code, SQL or actions found in documents or prompts (permanently excluded).
- SSO / OIDC / SAML, MFA, password-reset e-mail, SCIM; legal or certification-level compliance guarantees (SOC 2, ISO 27001, GDPR attestations).
- Relation-type extraction for the knowledge graph (edges are sentence-level co-mentions); real-time collaborative editing.
- Horizontal scaling of document storage without a shared volume or object store (see Risks).

6. Input Formats

Every extracted unit carries a location so that citations, filters and exact-search results can point to the original source.

Format	Handling	Evidence preserved	Status
PDF	Per-page text (pypdf); OCR fallback for scanned pages (Tesseract, MKA_OCR_ENABLED); bookmarks become section metadata; ruled tables extracted structurally (pdfplumber); page cap.	file name, document ID, page, section (when bookmarks exist)	**Implemented, tested** OCR tested with a generated scanned PDF
CSV / Excel (.xlsx)	Row units for search; full table kept for analysis; header = first non-empty row; encodings and delimiters sniffed.	file name, sheet, spreadsheet row number, column names	**Implemented, tested**
DOCX	Paragraphs and tables in document order; headings tracked as sections; macros rejected.	file name, paragraph / table + row, section	**Implemented, tested**
TXT / Markdown	Paragraph blocks; Markdown headings tracked.	file name, line, section	**Implemented, tested**
JSON	Leaf values flattened to JSON paths.	file name, JSON path	**Implemented, tested**
Public web URL	SSRF-guarded fetch (resolve once, connect by IP, verified SNI), readable-text extraction, HTML tables, canonical URL kept.	URL, title, section	**Implemented, tested** TLS pinning tested with a local TLS server
Images (PNG/JPEG/TIFF/WebP)	OCR text; optional caption from a vision-capable LLM (MKA_VISION_CAPTIONS).	file name, 'image (OCR)' / 'image caption'	**Implemented, partly tested** OCR tested; captions tested with a scripted model only

Validation before parsing: extension allow-list, magic-byte / container checks (PDF header, OOXML structure, image signatures, no binary data in text formats), zip-bomb limits, macro rejection, size limit, PDF page limit, filename sanitisation.

7. Functional Requirements

Priority uses MoSCoW. FR-01 to FR-14 carry over from v1 (strengthened); FR-15 to FR-30 are new in v2.

ID	Requirement	Description	Pri.	Status
FR-01	File upload	Accept supported files, validate type/size, return per-file ingestion status; multi-file; synchronous or queued.	Must	**Implemented, tested**
FR-02	Web ingestion	Public HTTP(S) URL with canonical URL retained; private/local/redirect-to-private targets blocked; DNS-rebinding-safe.	Must	**Implemented, tested**
FR-03	Content extraction & metadata	Pages, sheets/rows, sections, URLs, document IDs preserved for every unit; tables extracted as structured data.	Must	**Implemented, tested**
FR-04	Chunking & indexing	Chunks never cross page/sheet/table/section boundaries; embeddings stored in the configured vector store with tenant/workspace scope.	Must	**Implemented, tested**
FR-05	Exact search	Literal name/ID/phrase search over the complete extracted corpus; documented matching rules; total count, file, page numbers, excerpts; not top-K limited.	Must	**Implemented, tested**
FR-06	Semantic search	Vector retrieval of conceptually relevant passages with source excerpts.	Must	**Implemented, tested**
FR-07	Structured data analysis	Controlled pandas operations: filter, sort/top-N, group, sum, mean, median, min/max, distinct, duplicates, two-table comparison; never executes generated code.	Must	**Implemented, tested**
FR-08	Multi-agent routing	Supervisor routes to Exact, Semantic, Document Search, Web, SQL/Pandas, Summarization agents; rules by default, optional validated LLM tool-calling.	Must	**Implemented, tested**
FR-09	Evidence-based answers	Answers carry citations and excerpts; insufficient evidence yields an explicit 'could not verify' (verified=false).	Must	**Implemented, tested**
FR-10	Cross-source questions	Per-source retrieval and side-by-side comparison; LLM match/conflict analysis when configured.	Should	**Implemented, partly tested** LLM conflict analysis tested with a scripted model; side-by-side evidence tested for real.
FR-11	Conversation context	Per-conversation memory (window + rolling summary), follow-up resolution, isolation between users.	Should	**Implemented, tested**
FR-12	Error handling	Clear errors for unsupported/corrupt/empty files, invalid URLs, provider errors, indexing failures; no partial ingestion.	Must	**Implemented, tested**
FR-13	Re-ingestion and removal	Re-index and delete a source; text, chunks, vectors and original removed consistently.	Should	**Implemented, tested**
FR-14	API access	Documented FastAPI endpoints (OpenAPI), typed schemas, request IDs.	Must	**Implemented, tested**
FR-15	Advanced RAG	Semantic, BM25 and hybrid search; query rewriting; multi-query; HyDE; metadata filtering; parent-child retrieval; contextual compression; reranking; corrective RAG - each configurable globally and per request.	Must	**Implemented, partly tested** All techniques tested; LLM-dependent ones (HyDE, LLM rerank/compress/grade) with a scripted model; cross-encoder not installed.
FR-16	Supervisor and agents	LangGraph StateGraph with typed state, conditional edges, tool registry (validated arguments), checkpointing, bounded retries, session memory; eight agents including Verification and Final Response.	Must	**Implemented, tested**
FR-17	Verification	Citation-marker, number-grounding, lexical-support and optional LLM-judge checks; bounded retries; verbatim fallback; output guard against prompt leaks and exfiltration URLs.	Must	**Implemented, tested**
FR-18	OCR	Scanned PDF pages and image files read with Tesseract; OCR'd pages are flagged in warnings.	Must	**Implemented, tested**

ID	Requirement	Description	Pri.	Status
FR-19	Document intelligence	Summaries (map-reduce), entity extraction (heuristic + verbatim-validated LLM), table extraction, question generation, follow-up suggestions, language detection, multilingual answering and cross-lingual retrieval.	Should	**Implemented, partly tested** LLM paths tested with a scripted model; heuristic entity extraction covers Latin-script names only.
FR-20	Knowledge graph / Graph RAG	Sentence-level entity co-occurrence graph per accessible documents; neighbour/path queries; optional retrieval expansion.	Could	**Implemented, tested**
FR-21	Multimodal and voice	Image OCR/captions; voice via browser speech recognition or server-side transcription.	Could	**Implemented, partly tested** Whisper transcription mock-tested; captions scripted; images inside PDFs not indexed.
FR-22	Streaming	Server-Sent Events: real graph progress steps, answer pieces, final structured response.	Should	**Implemented, partly tested** Answer text is streamed after generation; provider token streaming not implemented.
FR-23	Background ingestion	Durable jobs with progress, idempotent claim, bounded retries for transient failures, in-process or Redis workers.	Must	**Implemented, tested**
FR-24	Authentication & RBAC	JWT (bcrypt passwords, revocation), roles viewer/editor/admin, role changes effective immediately, last-admin protection.	Must	**Implemented, tested**
FR-25	Multi-tenancy & document ACL	Tenant+workspace isolation at storage, vector and graph level; 'private' documents visible to owner and admins only.	Must	**Implemented, tested**
FR-26	Audit logging	Append-only audit events with request ID, user, IP, outcome; question text never stored (hash + length only).	Must	**Implemented, tested**
FR-27	Rate limiting	Per-user/IP limits for ask, ingest, auth; shared across replicas via Redis; 429 with Retry-After.	Must	**Implemented, tested**
FR-28	Data deletion	Delete source, conversation, workspace; admin purge of all tenant data with typed confirmation.	Must	**Implemented, tested**
FR-29	Evaluation	Recall@K, Precision@K, nDCG, MRR, routing/count/data/abstention accuracy, faithfulness and relevancy (proxy and LLM-judge), RAGAS adapter, latency, tokens, cost; dashboard and reports; strategy comparison.	Must	**Implemented, partly tested**
FR-30	Web dashboard	React + TypeScript: login, multi-upload with progress, document management, chat with citations and filters, history, evaluation dashboard, admin.	Must	**Implemented, tested**

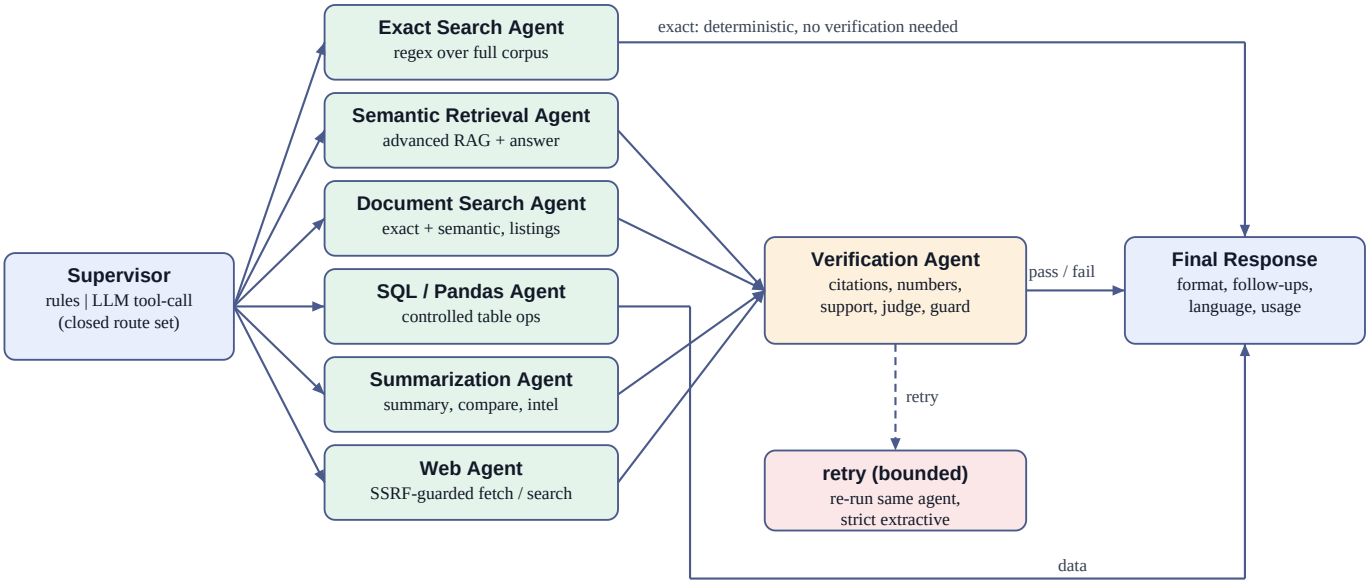
8. Non-Functional Requirements

ID	Area	Requirement	How it is met / verified
NFR-1	Accuracy	Answers grounded in retrieved or calculated evidence; exact counts scan the full relevant corpus; abstain when unverifiable.	Exact search is regex over all units (test: 40-page PDF count); verification agent; evaluation: exact-count accuracy 100% on the sample set.
NFR-2	Performance	Targets set per deployment from measurement, not assumed.	Sample corpus (offline, no LLM): p50 9 ms, p95 28 ms end-to-end. LLM-backed latency depends on the provider and was not measured.
NFR-3	Scalability	Batch processing; never place a whole large file in one prompt; asynchronous ingestion.	Chunking, map-reduce summaries, queue + workers (tested with Redis and a separate worker). BM25 index is rebuilt in memory per workspace - suited to tens of thousands of chunks; larger corpora need a search engine (Risk R-4).
NFR-4	Security	See section 13.	Automated tests per control.
NFR-5	Privacy	No cross-tenant or cross-user exposure; retention and deletion controls; no document text in logs or audit.	Isolation tests across tenants, workspaces, private documents, conversations, graph; purge test.
NFR-6	Reliability	Failures visible and retryable where safe; partial ingestion never shown as complete.	Atomic stage-and-rename commit; job retry policy; idempotent job claim; tests for outage/retry.
NFR-7	Maintainability	Modular adapters, typed schemas, tests, configuration-driven providers and stores.	Pydantic everywhere; ruff clean; strict TypeScript; provider/vector/queue/checkpoint backends behind interfaces.
NFR-8	Observability	Structured logs, request IDs, usage and cost tracking, audit trail.	JSON logs with request ID; per-request token/cost metering; /health usage counters; audit API.
NFR-9	Portability	Run locally without external services; run in containers with PostgreSQL, Redis, Qdrant.	Defaults need no keys (SQLite, local vectors, hash embedder). Compose file validated syntactically; image build not executed (no container runtime).
NFR-10	Usability	Accessible, responsive dashboard; clear errors; no HTML injection from documents.	ARIA labels, responsive CSS; XSS test; keyboard-operable upload.

9. Multi-Agent Architecture

A LangGraph **StateGraph** coordinates specialised agents. State is typed and JSON-serialisable so it can be **checkpointed** (memory, SQLite or PostgreSQL) per conversation thread; heavy objects (principal, stores) live in a per-request context, never in the checkpoint.

LangGraph StateGraph - typed, checkpointed state (memory | SQLite | PostgreSQL)



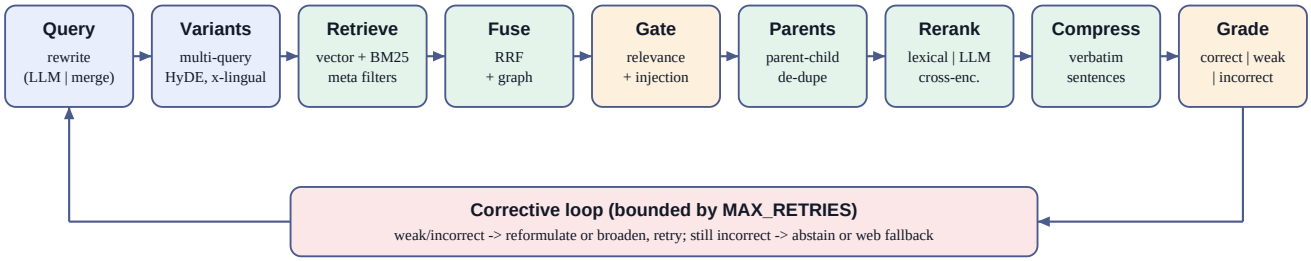
semantic -> Web Agent when abstained and web fallback is enabled; data -> Semantic Agent when the question is not about the tables

Agent	Responsibility	Tools it may use	Verified
Supervisor	Chooses the route from the question and available sources (rules; optional LLM tool-calling restricted to a closed set, with rules as fallback).	routing rules / closed route set	route table test; LLM mode: valid, invalid and disallowed choices
Exact Search Agent	Literal name/ID/phrase search over the whole extracted corpus; count, files, pages, excerpts; asks for the term when missing.	exact_search	count/page tests; evaluation 100%
Semantic Retrieval Agent	Runs the advanced RAG pipeline, builds a grounded answer (LLM or verbatim extraction), abstains when evidence is missing.	hybrid_search	recall/MRR on sample set; abstention
Document Search Agent	Which documents mention X, clause lookups, document listing; combines exact and semantic evidence.	exact_search, hybrid_search, list_sources	clause 8 vs 80 vs 8.1 tests
SQL / Pandas Agent	Turns a question into a whitelisted plan (heuristic or validated LLM JSON) and executes it with pandas; asks for clarification when ambiguous; hands over when the question is not about the tables.	table_query	aggregate tests vs manual calculation
Summarization Agent	Map-reduce summaries; cross-document comparison; entity, table and question extraction; knowledge-graph questions.	summarize, hybrid_search	summary/compare/entity/graph tests
Web Agent	Reads URLs given in the question or search results (SSRF-guarded); transient evidence, never auto-indexed; optional fallback when documents cannot answer.	fetch_url, web search provider	mock-transport tests; SSRF tests
Verification Agent	Checks citations, numbers, lexical support, optional LLM judge, output guard; decides pass / retry (bounded) / fail (verbatim fallback).	-	retry, exhaustion, judge, guard tests
Final Response Agent	Assembles the response: citations, limitations, confidence, follow-up questions, language, trace, usage.	-	API/e2e tests

Other behaviour: bounded retries (MAX_RETRIES) in both corrective RAG and verification; session memory = last N turns plus a rolling summary of older turns; every node degrades to an honest message on internal error instead of a 500; tool arguments are validated (Pydantic) and there is no tool that executes code, SQL or shell commands.

10. Advanced RAG Architecture

The pipeline is configurable globally (environment) and per request (`retrieval` object). Every step is recorded in a trace for debugging and evaluation.



Technique	How it works here	Config key	Default	Without an LLM
Semantic search	Cosine over embeddings (local, Chroma or Qdrant); tenant/workspace filter mandatory.	RETRIEVAL_MODE	hybrid	hash embedder is lexical, not semantic
BM25 / keyword	Okapi BM25 with inverted index; fuzzy prefix expansion on corrective passes.	RETRIEVAL_MODE	hybrid	works
Hybrid	Reciprocal-rank fusion of every (query variant x channel) ranking.	RETRIEVAL_MODE	hybrid	works
Query rewriting	LLM standalone-query rewrite from conversation; otherwise follow-up merge with previous question.	QUERY_REWRITE	on	context merge
Multi-query	LLM paraphrases (incl. other languages); otherwise keyword-only variant.	MULTI_QUERY	off	keyword variant
HyDE	Embeds an LLM-written hypothetical answer for vector search (never shown).	HYDE	off	skipped (reported in trace)
Metadata filtering	kinds, name, page range, sheet, section, ingestion date; applied before ranking.	retrieval.filters	none	works
Parent-child	Small chunks are matched; their page/section parent is returned; siblings de-duplicated.	PARENT_CHILD	on	works
Contextual compression	Keep only query-relevant sentences, verbatim (LLM choices must literally occur in the source).	COMPRESSION	on	extractive
Reranking	Lexical (coverage, BM25, phrase), LLM scoring, or optional cross-encoder.	RERANKER	lexical	lexical
Corrective RAG	Grade evidence; on weak/no evidence reformulate or broaden and retry (bounded), else abstain or web fallback.	CORRECTIVE, MAX_RETRIES	on, 2	heuristic grading
Graph RAG	Adds chunks linked to query entities and their strongest neighbours.	GRAPH_RAG	off	works
Cross-lingual	LLM translates the question into the languages of the indexed documents.	CROSS_LINGUAL	on	skipped

Safety in the pipeline: passages sent to an LLM are fenced as untrusted data with neutralised control markup; suspected injections are flagged (or excluded with INJECTION_POLICY=exclude); every LLM output is validated and failures fall back to deterministic behaviour.

11. Platform and Data Architecture

Component	Role	Notes
FastAPI	REST + SSE API, OpenAPI docs, static React dashboard	request IDs, JSON logs, security headers, CORS allow-list
PostgreSQL	Tenants, users, revoked tokens, audit events, jobs, conversations/messages, evaluation runs; LangGraph checkpoints	SQLite for development; schema created at start (no migration tool yet - Risk R-6)
Qdrant (production) / ChromaDB (dev) / local	Vector index. Qdrant: one collection per embedding dimension; tenant, workspace and source in the payload; mandatory filters on every search and delete	server mode not tested here; embedded mode and the isolation contract were
Redis	Shared rate limiting; ingestion job queue (reliable move to a processing list, crash re-delivery)	falls back to in-process when not configured
Worker	`python -m app.worker`: executes ingestion jobs; shares the data volume with the API	atomic claim makes duplicate delivery harmless
Document store	Per source: original file, units, chunks, vectors (local mode), tables, graph, metadata; staged then atomically renamed	filesystem volume; object storage is future work (Risk R-3)
Tenancy	Address = tenant / workspace / source; no API crosses tenants; private documents filtered below the API	X-Session-ID selects the workspace within the tenant
React dashboard	Single-page app served by the API	JWT kept in sessionStorage (Risk R-7)

12. API Requirements

Full reference: `docs/API.md` (generated from the application), interactive Swagger UI at `/docs`. Conventions: bearer JWT, `X-Session-ID` workspace header, `X-Request-ID` correlation, JSON errors, `429` with `Retry-After`.

Area	Endpoints	Minimum role
System	GET /health	public
Auth	POST /auth/signup (dev only), /auth/login, /auth/logout; GET /auth/me	public / any
Sources	POST /sources/upload[?async=true], /sources/url, /sources/{id}/reindex; GET /sources, /sources/{id}, /sources/{id}/tables|entities|questions; DELETE /sources/{id}	viewer read; editor write; owner/admin modify
Jobs	GET /jobs, /jobs/{id}	editor
Chat	POST /ask, /ask/stream (SSE); GET/POST /conversations, GET /conversations/{id}/messages|trace; DELETE /conversations/{id}; POST /transcribe	viewer
Intelligence	GET /graph (top entities, neighbours, path)	viewer
Evaluation	POST /eval/run; GET /eval/runs, /eval/runs/{id}, /report.html, /report.md	editor
Admin	GET/POST /admin/users, PATCH/DELETE /admin/users/{id}; GET /admin/audit, /admin/settings; DELETE /admin/data, /session	admin

/ask response: answer, route, confidence (high|medium|low|none), verified, citations[{source, location, excerpt, score}], limitations, needs_input, follow-up questions, language, usage (tokens, cost), latency, details (plan, verification, retrieval trace, counts, table results).

13. Security Requirements

Threat	Controls	Verification	Status
Prompt injection in documents or questions	Ingestion-time scan; fenced untrusted data with neutralised markup; closed tool registry (no code/SQL/shell); output guard (prompt-leak, exfiltration URLs); optional exclude policy; LLM numbers/citations verified	tests: pattern detection and false-positive set, fence forgery, guard, injected document end-to-end	**Implemented, tested**
SSRF and DNS rebinding	Scheme/credential checks; resolve once; reject non-public addresses (incl. IPv4-mapped IPv6); connect by vetted IP with verified SNI; every redirect re-validated; size/time/type limits	tests: private ranges, metadata endpoint, redirect-to-private, rebinding with a local TLS server and wrong-certificate rejection	**Implemented, tested**
Malicious or malformed files	Extension allow-list; magic bytes; OOXML structure; zip-bomb and macro rejection; binary-in-text detection; size and page limits; safe filenames	tests for each rejection	**Implemented, tested**
Authentication	bcrypt, constant-time-ish login, JWT with required claims, revocation list, disabled-account and role changes effective immediately, `alg=none`/forged/expired tokens rejected	auth test-suite	**Implemented, tested**
Authorisation and tenant isolation	RBAC per route; tenant+workspace addressing; mandatory Qdrant filters; ACL-filtered graph and retrieval; hidden documents look missing	cross-tenant, private-document, conversation, job, graph and evaluation isolation tests	**Implemented, tested**
Abuse / DoS	Per-identity rate limits (memory or Redis); upload and PDF page limits; bounded retries; request timeouts	rate-limit tests incl. two replicas via Redis	**Implemented, tested**
Secrets	Keys only from environment; SecretStr; masked admin settings; production start-up validator rejects insecure config	config validation and masking tests; production smoke	**Implemented, tested**
Auditability	Append-only events with request ID, user, IP, outcome; no document or question text	audit content/privacy tests	**Implemented, tested**
Data deletion / retention	Delete source/conversation/workspace; tenant purge; no automated retention schedule yet	deletion and purge tests	**Implemented, partly tested**
XSS in the dashboard	All document/answer text rendered as text; CSP and security headers	Vitest injection test, Playwright run with no console errors	**Implemented, tested**

Residual risks: no SSO/MFA; JWT in browser sessionStorage; rate limiting trusts the direct client address unless a proxy sets it; heuristic injection scanning cannot catch every phrasing (defence in depth, not a guarantee); 'X-Forwarded-For' is not trusted by default.

14. Evaluation Metrics

The harness (`python -m app.eval`, `POST /eval/run`, dashboard tab) runs a labelled dataset through the real pipeline and agents and reports:

Metric	Definition
Recall@K	share of gold passages found in the top K retrieved (K = 1, 3, 5)
Precision@K	relevant results in the top K divided by K
MRR	mean of 1/rank of the first relevant result
nDCG@K	binary-relevance discounted gain
Faithfulness	*proxy*: share of answer sentences whose content words and numbers occur in the cited evidence; LLM-judge variant when an LLM is configured; RAGAS adapter available
Answer relevancy	*proxy*: cosine(question, answer) under the configured embedder; LLM-judge variant when configured
Routing / count / data accuracy	expected route, exact occurrence count, numeric result match
Abstention accuracy	unanswerable questions must return verified=false (or low confidence, reported separately)
Latency	end-to-end per question: mean, p50, p95; retrieval-only p50/p95
Tokens and cost	provider-reported when available else ~4 chars/token; USD from MKA_PRICE_TABLE

Measured on the built-in synthetic corpus (42 questions over 8 mixed-format documents; offline: hash embedder, no LLM; run 2026-10-30T05:42:51Z):

Metric	Value	Metric	Value
Recall@1 / @3 / @5	90.9% / 90.9% / 90.9%	Routing accuracy	100.0%
Precision@1 / @3 / @5	90.9% / 30.3% / 18.2%	Exact-count accuracy	100.0%
MRR	0.909	Data (table) accuracy	100.0%
Faithfulness (proxy)	88.9%	Abstention accuracy (strict)	75.0%
Answer relevancy (proxy)	26.7%	Keyword correctness (semantic)	92.3%
Latency p50 / p95	9 / 28 ms	Tokens / cost per query	0 / $0.00000

Strategy comparison (same corpus):

Strategy	Recall@5	MRR	nDCG@5	p50 ms
semantic	86.4%	0.833	0.841	7.3
keyword (BM25)	86.4%	0.864	0.864	7.8
hybrid	86.4%	0.833	0.841	8.8
hybrid + lexical rerank	86.4%	0.864	0.864	8.1
hybrid + rerank + parent-child + corrective	90.9%	0.909	0.909	8.6
hybrid + multi-query + graph	90.9%	0.909	0.909	11.1

How to read these numbers. The corpus is small and synthetic; it detects regressions and compares settings, it is not a benchmark of real-world accuracy. With the offline hash embedder the semantic and keyword channels are both lexical, so the hybrid advantage appears only with a real embedding model. Hard paraphrases with no word overlap (e.g. 'vacation time' vs 'annual leave') fail by design and are listed in the report's failures. The proxy metrics are not RAGAS; LLM-judge and RAGAS values require a configured model and were not measured here.

15. Acceptance Criteria

ID	Criterion	Status
AC-1	A text PDF is ingested and searched; returned matches carry the correct page numbers.	**Implemented, tested**
AC-2	A scanned PDF is rejected with a clear message without OCR and indexed with OCR; OCR'd pages are flagged.	**Implemented, tested**
AC-3	CSV/XLSX aggregates (sum, mean, median, max, min, group, filter, sort, duplicates, comparison) equal a trusted manual calculation; ambiguous questions trigger a clarification instead of a guess.	**Implemented, tested**
AC-4	An exact-name query returns the total count, file names, page numbers and excerpts over the complete corpus (40-page test), with documented rules (case, whole word, clause 8 vs 8.1 vs 80).	**Implemented, tested**
AC-5	A public page is ingested and cited by URL; private, local, metadata-service and redirect-to-private URLs are rejected; DNS rebinding cannot redirect the connection.	**Implemented, tested**
AC-6	A semantic question returns relevant passages with citations; an unrelated question returns an explicit 'could not verify' (verified=false).	**Implemented, tested**
AC-7	A generated answer containing an invented number or no citations is retried (bounded) and finally replaced by verbatim evidence.	**Implemented, tested**
AC-8	Users of tenant A cannot list, read, search, cite, graph or delete anything of tenant B; private documents are invisible to other users in the same tenant; role limits are enforced.	**Implemented, tested**
AC-9	Ingestion runs in the background with visible progress; transient failures are retried at most N times; invalid input is not retried; a separate Redis worker processes jobs.	**Implemented, tested**
AC-10	Rate limits return 429 with Retry-After and are shared by multiple API replicas through Redis.	**Implemented, tested**
AC-11	The audit log records ingest, ask, login, permission denials and deletions without storing question or document text.	**Implemented, tested**
AC-12	Deleting a source or purging a tenant removes files, chunks and vectors; deleted material is no longer retrievable.	**Implemented, tested**
AC-13	The evaluation harness reports Recall@K, Precision@K, MRR, faithfulness, relevancy, latency, tokens and cost, and fails CI when retrieval quality drops below the regression thresholds.	**Implemented, tested**
AC-14	The dashboard supports multi-upload with progress, document management, chat with citations and filters, history, evaluation, admin; document text cannot inject HTML.	**Implemented, tested**
AC-15	With a real LLM and embedding provider, answer quality, faithfulness (judge/RAGAS) and cost meet targets agreed with the business owner on a representative corpus.	**Pending**
AC-16	The container images build and the compose stack starts with PostgreSQL, Redis and a Qdrant server; Qdrant server mode passes the vector-backend contract tests.	**Pending**

16. Development Roadmap

Phase	Deliverables	Exit condition	State
1 Platform foundation	PostgreSQL model, JWT auth, RBAC, tenant-scoped storage, document ACL, rate limiting, audit, purge, production config validation	isolation and RBAC tests pass (incl. PostgreSQL, Redis)	**Implemented, tested**
2 Ingestion	File validation, OCR, PDF sections/tables, images, DNS-rebinding-safe fetch, background jobs, Qdrant backend	format, security and job tests pass	**Implemented, tested**
3 Advanced RAG	Hybrid, rewrite, multi-query, HyDE, filters, parent-child, compression, rerank, corrective	technique tests and trace assertions pass	**Implemented, tested**
4 Agents	Supervisor + 8 agents, tool registry, checkpointing, verification, memory, streaming	routing, verification, memory, checkpoint tests pass	**Implemented, tested**
5 Intelligence	Data ops, entities, tables, questions, multilingual, knowledge graph, voice	operation and isolation tests pass	**Implemented, tested**
6 Evaluation	Metrics, harness, reports, dashboard API, regression thresholds	sample evaluation within thresholds	**Implemented, tested**
7 Frontend	React dashboard, unit and end-to-end tests	e2e smoke passes against the real server	**Implemented, tested**
8 Delivery	Docker, compose, CI workflow, documentation, this BRD	documents produced; container build pending	**Implemented, partly tested** image build and CI run not executed here
Next A	Validate with real providers: embeddings, LLM, RAGAS, cross-encoder; tune thresholds; cost budget	AC-15	**Pending**
Next B	Run on the container runtime and Qdrant server; Alembic migrations; object storage; OIDC/SSO and MFA; refresh tokens; retention scheduler; provider token streaming	AC-16 and hardening backlog	**Pending**

17. Risks and Dependencies

ID	Risk	Impact	Mitigation / status
R-1	Offline default embedder is lexical; paraphrase recall is limited	missed passages on conceptual questions	configure OpenAI/Gemini embeddings; evaluation reports failures; hard cases tracked
R-2	LLM-dependent features were tested with scripted models only	unknown real-world quality/cost	AC-15; evaluation harness with LLM judge and RAGAS; verification agent bounds damage
R-3	Documents live on a filesystem volume shared by API and worker	horizontal scaling and DR need shared storage	volume today; object storage on the roadmap
R-4	BM25 index is built in memory per workspace	memory/latency on very large corpora	OpenSearch / PostgreSQL full-text search planned
R-5	OCR quality depends on scans; images inside PDFs not indexed	missed content	OCR flagged in warnings; roadmap item
R-6	No migration tool (schema created at start)	upgrades need manual care	adopt Alembic before first production upgrade
R-7	JWT in sessionStorage; no SSO/MFA	token theft if XSS ever occurs	strict CSP, text-only rendering, short token life; OIDC + httpOnly cookies planned
R-8	Heuristic prompt-injection detection is evadable	residual injection risk	defence in depth: data fences, closed tools, output guard, verification, exclude policy
R-9	Embedding/LLM usage cost	operating cost	per-request token/cost metering, rate limits, price table, evaluation of cost per query
R-10	Web pages change or block extraction	stale or missing web evidence	URL and ingestion time retained; re-index re-fetches
R-11	Qdrant server, Docker build and CI were not executed in the build environment	integration surprises	AC-16; compose file validated; embedded Qdrant passes the same contract tests

Dependencies: Python 3.12, PostgreSQL 16, Redis 7, Qdrant (production), Tesseract OCR binary, LLM/embedding provider credentials supplied through the environment, Node 22 for the dashboard build.

Appendix A - Verification limits (what was NOT tested)

- Live OpenAI / Gemini calls (chat, embeddings, vision, transcription): no credentials in the build environment. LLM-dependent code paths were exercised with scripted test doubles that verify orchestration, validation and fallbacks - not model quality.
- RAGAS: adapter plumbing tested against a stub module only. Cross-encoder reranker: dependency not installed; the fallback path is tested.
- Qdrant server mode: the server binary could not be executed in the sandbox; embedded mode passes the vector-backend contract and tenant-filter tests.
- Docker image build and docker compose start-up: no container daemon; the compose file parses and validates. GitHub Actions workflow: syntax-valid but not run; each command in it was run locally.
- Browser speech recognition and microphone capture; server-side Whisper (HTTP mocked).
- Load, soak and penetration testing; behaviour on corpora larger than the evaluation set.

Appendix B - Traceability from BRD v1

v1 requirement	v2 treatment	State
FR-01 File upload	FR-01 + validation, async jobs, visibility	**Implemented, tested**
FR-02 Web ingestion	FR-02 + DNS-rebinding-safe connect-by-IP	**Implemented, tested**
FR-03 Extraction	FR-03 + sections, tables, OCR	**Implemented, tested**
FR-04 Chunking/indexing	FR-04 + parent spans, tenant-scoped vectors	**Implemented, tested**
FR-05 Exact search	FR-05 unchanged semantics; page lists added	**Implemented, tested**
FR-06 Semantic search	FR-06 + FR-15 advanced RAG	**Implemented, tested**
FR-07 Structured data	FR-07 extended (sort, median, distinct, duplicates, comparison)	**Implemented, tested**
FR-08 Routing	FR-08 + FR-16 eight-agent supervisor	**Implemented, tested**
FR-09 Evidence-based answers	FR-09 + FR-17 verification	**Implemented, tested**
FR-10 Cross-source	FR-10	**Implemented, partly tested** conflict analysis needs an LLM
FR-11 Conversation context	FR-11 + persistent memory with rolling summary	**Implemented, tested**
FR-12 Error handling	FR-12	**Implemented, tested**
FR-13 Re-ingestion/removal	FR-13 + purge	**Implemented, tested**
FR-14 API access	FR-14 + OpenAPI + SSE	**Implemented, tested**

Appendix C - Configuration reference

All settings are environment variables (prefix `MKA_`; provider keys `OPENAI_API_KEY`, `GEMINI_API_KEY`, `BRAVE_API_KEY`). The complete annotated list is in `.env.example`. Production mode (`MKA_ENV=production`) refuses to start unless authentication is enabled, a 32+ character JWT secret is set, self-service signup and private-URL fetching are off, CORS has no wildcard and the database is PostgreSQL.

Appendix D - Delivered artefacts

- Source code: `app/` (API, agents, RAG, ingestion, security, intelligence, evaluation), `frontend/` (React + TypeScript), `tests/`, `scripts/`.
- Deployment: `Dockerfile`, `docker-compose.yml`, `.github/workflows/ci.yml`, `.env.example`.
- Documentation: `README.md`, `docs/API.md`, `docs/openapi.json`, `docs/eval/` (latest evaluation report in JSON, Markdown and HTML).